

☒ 日記自動化セッション記録

Session Summary — 2026-06-29 / 2026-06-30

本ドキュメントは、StuMane リポジトリにおける日記PDF自動生成ワークフロー構築セッション（Claude Code × GitHub Actions）の全記録です。実装内容・設計判断・残タスク・コードレビュー所見をまとめています。

1. セッション概要

ユーザーが既に Notion Claude Brain で設計した日記作成フローを完全自動化（人手ゼロ）することが目的。毎朝・毎週・毎月の PDF を GitHub Actions で生成し、iOS ショートカット経由で GoodNotes に取り込む仕組みを構築した。

種別	スケジュール	出力ファイル名
デイリー	毎朝 05:00 JST	diary-YYYY-MM-DD-daily.pdf
ウィークリー	毎週日曜 22:00 JST	diary-YYYY-MM-week-N-weekly.pdf
マンスリー	毎月1日の2日前 05:00 JST	diary-YYYY-MM-monthly.pdf

2. 技術スタック

レイヤー	ライブラリ / サービス	用途
スケジューラ	GitHub Actions (cron)	定期実行
PDF生成	ReportLab + pypdf	overlay合成
フォント	IPA Gothic (ipag.ttf)	日本語描画
テンプレート	MUJI_Notebook_2026-2028.pdf	無印良品風ノート
Notionタスク	Notion REST API v1	タスク総括DB取得
カレンダー	caldav + icalendar + pytz	iCloud CalDAV
Git自動コミット	stefanzweifel/git-auto-commit-action@v5	PDF保存
GoodNotes連携	iOS ショートカット	GitHub raw→GoodNotes

3. 実現方式の選定プロセス

以下の4案を比較し、案D（GitHub Actions + iOS ショートカット）を採用。

案	名称	特徴	採否
A	Gmail経由	メール添付→GoodNotes 簡単だがPDF劣化リスク	×

B	Markdown変換	GoodNotes MCPで作成 日本語レイアウト困難	×
C	iCloud Drive	ショートカットで配置 手動操作が残る	×
D	GitHub Actions +iOS Shortcut	完全自動・PDF品質保持 月額コストなし	✓

※ GoodNotesのMCPはmarkdown/SVG/Mermaid作成のみ対応。PDFインポート・ページ移動は不可のため、「日記（まとめ）へのページ移動」要件は取り下げ。

4. 実装アーキテクチャ

4-1. GitHub Actions ワークフロー

.github/workflows/diary-daily.yml（デイリー + マンスリー兼用）

```
cron: '0 20 * * *' ← UTC 20:00 = JST 翌 05:00
```

generate_daily.py と generate_monthly.py
を逐次実行。月次スクリプトは当日が「1日の2日前」でなければ自己スキップ。

.github/workflows/diary-weekly.yml（ウィークリー）

```
cron: '0 13 * * 0' ← UTC 13:00 日曜 = JST 22:00 日曜
```

両ワークフロー共通設定：

- permissions: contents: write
- stefanzweifel/git-auto-commit-action@v5 で diary/output/*.pdf を自動コミット
- secrets: NOTION_TOKEN / APPLE_ID / APPLE_APP_PASSWORD

4-2. テンプレート PDF とページマッピング

diary/template/MUJI_Notebook_2026-2028.pdf (137.6KB, 27ページ)

ページIndex	用途
0	表紙（変更なし）
1	デイリー（generate_daily.py）
2	ウィークリー（generate_weekly.py）
3～26	マンスリー（2026/4→index3, 2028/3→index26）

☒ MONTH_PAGE のインデックスを1ずらすと別の月に描画される。（2026/4 → index3 が基準）

4-3. Pythonスクリプト一覧

ファイル	主な処理
fetch_notion.py	Notion API でタスク総括DB取得 (block2=自分タスク, block3=依頼タスク)
fetch_calendar.py	iCloud CalDAV でカレンダーイベント取得 JSTに変換して返却
generate_daily.py	テンプレートpage[1]にオーバーレイ 時刻付き予定 + 終日予定 + Notionタスク

generate_weekly.py	テンプレートpage[2]にオーバーレイ 翌週(月～日)の予定 + Notionタスク
generate_monthly.py	テンプレートpage[pidx]にオーバーレイ 月内予定 + Notionタスク (自己スキップあり)

5. iOS ショートカット設定（ユーザー作業）

GitHub Actions が PDF をコミット後、iOS ショートカットで自動ダウンロード → GoodNotes に追加。

ショートカット名	起動時刻	GitHub Raw URL (末尾のファイル名部分)
Daily Diary	05:10 JST 毎日	diary- <code>{YYYY}</code> - <code>{MM}</code> - <code>{DD}</code> -daily.pdf
Weekly Diary	22:10 JST 日曜	diary- <code>{YYYY}</code> - <code>{MM}</code> -week- <code>{N}</code> -weekly.pdf
Monthly Diary	05:10 JST 毎月29日	diary- <code>{YYYY}</code> - <code>{MM}</code> -monthly.pdf

ショートカットの手順: 「URLの内容を取得」 → 「GoodNotesで開く」 (Open In アクション)

ベース URL: <https://raw.githubusercontent.com/weekly-from3rd-party/StuMane/main/diary/output/>

6. 残タスク（ユーザー側作業）

☐ 1. GitHub Secrets 登録

Settings → Secrets and variables → Actions に以下を追加: NOTION_TOKEN / APPLE_ID /
APPLE_APP_PASSWORD (アプリ用パスワード)

☐ 2. ブランチのマージ

claude/recurring-task-automation-lc0s4n → main へプルリクエスト&マージ

☐ 3. workflow_dispatch でテスト実行

Actions タブから手動実行し、PDF が diary/output/ にコミットされることを確認

☐ 4. iOS ショートカット作成

上記 5節 の3つのショートカットを Personal Automation で設定

☐ 5. Hello Weekly 3 の作業再開

別セッションで継続 (本セッションでは途中停止)

7. コードレビュー所見（9件）

高工数 (high) レビューで確認/推定の9件を報告。重要度順に列挙。

[CRITICAL] C1 generate_daily.py:69

DTEND が None のとき e.hour でクラッシュ

CalDAV の DTEND が省略されたイベントが存在すると AttributeError。修正: e.hour → (e.hour if e else 0)
または事前 None チェック。

[HIGH] C2 `fetch_notion.py:32`

Notion API ページネーション未実装

`has_more=True` でも `start_cursor` を使った追加フェッチをしないため、タスク101件目以降がサイレントに欠落する。

[HIGH] C3 `fetch_notion.py:57`

締切フィルタが過去分を永遠に蓄積

`dead_start <= target_str` の条件で歴史的に期限切れのタスク全件がマッチし続ける。「本日締切のみ」とする場合は `==` に変更。

[HIGH] C4 `fetch_calendar.py:27`

CalDAV 接続失敗を無音で [] 返却

CalDAV エラー時に空リストを返し、呼び出し元が空白 PDF をコミットしてしまう。raise を追加し、ワークフローを失敗させるべき。

[MED] C5 `generate_monthly.py:56 / generate_weekly.py:54`

end_dt が 23:59 で末尾イベントを取りこぼす可能性

00:00~23:59 ではなく翌日 00:00 (exclusive) にすると RFC 4791 CalDAV 準拠。

[MED] C6 `fetch_calendar.py:69`

終日イベントの DTEND が exclusive のまま格納

RFC 5545 では終日イベントの DTEND は翌日 0:00 (exclusive)。last_day_of_month に表示したい場合は -1day が必要。

[LOW] C7 `generate_weekly.py:40`

月曜に workflow_dispatch すると翌週翌週のPDFを生成

$(7 - \text{today.weekday()}) \% 7 \text{ or } 7$ の式は月曜(weekday=0)で 7 を返す。手動実行を今週分にしたい場合は別ロジックが必要。

[LOW] C8 `diary-daily.yml:5`

コメントの「前日」が誤解を招く

'0 20 * * *' は UTC 20:00 = JST 翌日 05:00。「前日」ではなく「当日未明」と書くと正確。

[LOW] C10 `fetch_calendar.py:35`

カレンダー単位のエラーがサイレントに飲み込まれる

per-calendar の date_search 例外を print だけで無視するとどのカレンダーが欠落したか追跡不能。ログレベルの改善を推奨。

8. Notion 連携メモ

本セッションで更新した Notion ページ:

- Chat Logs (38ad4ecd...) ← セッション記録エントリを追記
- Claude Brain (388d4ecd...) ← 近況メモ 2026-06-29 エントリを追記

タスク総括DB の DATABASE_ID:

f6ccdaa7-0c50-4f33-9a1c-31dd3819054e

9. 追加されたファイル構成

```
diary/ ├── output/ | └── .gitkeep # 出力ディレクトリ保持用 ├── scripts/ | └── requirements.txt #
reportlab pypdf caldav icalendar pytz requests | └── fetch_notion.py # Notion タスク取得 | └──
fetch_calendar.py # iCloud CalDAV カレンダー取得 | └── generate_daily.py # デイリーPDF生成 | └──
generate_weekly.py # ウィークリーPDF生成 | └── generate_monthly.py # マンスリーPDF生成 (自己スキップ付)
└── template/ └── MUJI_Notebook_2026-2028.pdf # 27ページ テンプレート .github/workflows/ ├──
diary-daily.yml # cron 0 20 * * * (JST 05:00 毎日) └── diary-weekly.yml # cron 0 13 * * 0 (JST 22:00 日曜)
```